TABLA DE CASOS

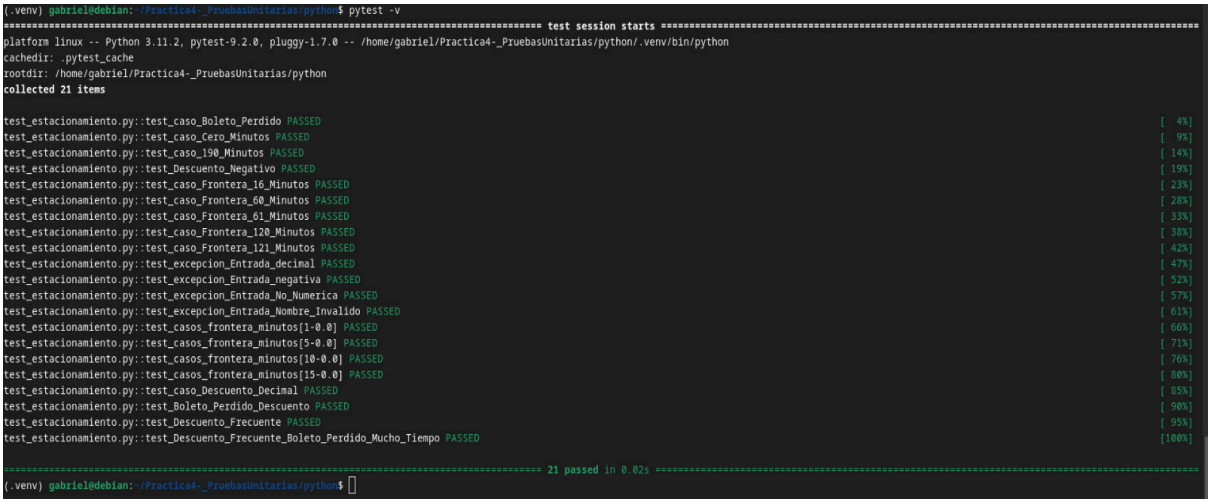
Caso	Entrada	Resultado esperado	Regla(s)	Justificación
1	minutos: 1.1 cliente: “normal” perdido: “false”	Lanzar Error	R1	**Entrada Invalida:** Los minutos no pueden no ser naturales y el programa debe lanzar un error.
2	minutos: -1 cliente: “normal” perdido: false	Lanzar Error	R1, R10	**Entrada Invalida**: Los minutos no pueden ser negativos y el programa debe lanzar un error.
3	minutos: 1 cliente: “anormal” perdido: false	Lanzar Error	R2, R11	**Entrada Invalida:** El tipo de cliente es invalido y el programa debe lanzar un error.
4	minutos: 1 cliente: “frecuente” perdido: true	Tarifa: $300.00	R3, R8	**Interacción entre reglas:** No debe existir descuento si el boleto se perdió.
5	minutos: 1 cliente: “normal” perdido: true	Tarifa: $300.00	R3	**Caso normal:** El boleto está perdido, se ignoran los demás datos y se pone la tarifa final.
6	minutos: 0 cliente: “normal” perdido: false	Tarifa: $0.00	R4	**Caso normal:** Como los minutos son cero y el boleto no está perdido, la tarifa es cero.
7	minutos: 15 cliente: “normal” perdido: false	Tarifa: $0.00	R5	**Caso frontera:** 15 minutos es el último minuto donde la tarifa es de cero.
8	minutos: 16 cliente: “normal” perdido: false	Tarifa: $20.00	R6	**Caso frontera:**16 minutos es el primer minuto donde entra la tarifa de $20.00.
9	minutos: 60 cliente: “normal” perdido: false	Tarifa: $20.00	R6	**Caso frontera:** 60 minutos es el último minuto donde la tarifa es de $20.00.
10	minutos: 61 cliente: “normal” perdido: false	Tarifa: $35.00	R7	**Caso frontera:** 61 minutos es el primer minuto donde entra la tarifa de $35.00.
11	minutos: 120 cliente: “normal” perdido: false	Tarifa: $35.00	R7	**Caso frontera:** 120 minutos es el último minuto donde la tarifa es de $35.00.
12	minutos: 121 cliente: “normal” perdido: false	Tarifa: $: 50.00	R7	**Caso frontera:** 121 minutos es el primer minuto donde entra la tarifa de $50.00.

13	minutos: 190 cliente: “normal” perdido: false	Tarifa: $65.00	R7	**Caso Normal:** 190 en minutos la tarifa es de $65.00.
14	minutos: 30 cliente: "frecuente" perdido: false	Tarifa: $18.00	R6, R8	**Interacción entre reglas:** 30 minutos es una tarifa de $20.00 y tiene un descuento de 10%.
15	minutos: 0 cliente: “frecuente” perdido: false	Tarifa: $0.00	R4, R8, R9	**Caso normal:** Aplicar 10% de descuento sobre $0.00 debe mantener la tarifa final en $0.00.
16	minutos: 90 cliente: “frecuente” perdido: false	Tarifa: $31.50	R8, R7	**Números decimales:**Operación con flotantes
17	minutos: 10000 cliente: “frecuente” perdido: “true”	Tarifa: $300.00	R3, R7,R8	**Equipo:** Se tienen varias consideraciones, y no se debe aplicar un descuento ni dar una tarifa mayor a $300.

- ## Reglas cubiertas:

Se hizo al menos una prueba con cada regla como se puede observar en la tabla de casos, por lo que los casos de las reglas considerados en ellos se cumplen.

- ## Evidencia de ejecución



- ## Respuestas de Análisis

1. ¿Qué diferencia hay entre un caso normal y un caso frontera?

Un caso normal prueba el comportamiento del sistema en situaciones típicas y seguras.

Un caso frontera prueba los límites exactos donde las reglas cambian de estado para asegurar que la transición entre comportamientos no falle.

2. ¿Cuál de sus pruebas considera más importante y por qué?

Considero que todas las pruebas son importantes, ya que cada una detecta fallos que no deberían ocurrir bajo ninguna circunstancia. Sin embargo, si tuviera que jerarquizarlas por orden de prioridad, sería el siguiente:

1. Las pruebas de excepciones y entradas inválidas: Son las más críticas porque evitar que el sistema colapse o lance un error no controlado (TypeError, Crash) es prioridad.
2. Los casos frontera: Porque son los puntos donde la lógica cambia de estado. Si una frontera falla, el sistema empieza a cobrar de más o de menos a muchísimos usuarios.
3. Casos normales e interacciones: Las pruebas de flujo común, que aseguran que el negocio calcule correctamente los descuentos y las tarifas del día a día.

3. Encontró algún comportamiento de la implementación que no coincida con el modelo?

Si, la implementación original contenía varios fallos graves respecto al modelo:

* R1 (Tipado): No valida que los minutos sean enteros, permitiendo decimales sin lanzar error, y si recibe texto arroja un TypeError no controlado en lugar de ValueError.
* R5 (Límite de gracia): Usaba `< 15` en lugar de `<= 15`, cobrando a los 15 minutos exactos cuando debía ser $0.00.
* R7 (Horas adicionales): Usaba la división entera `// 60`, fallando al cobrar la hora iniciada (61 min cobraba $20 en lugar de $35; 121 min cobraba $35 en lugar de $50).
* R8 (Descuento inapropiado): Le aplicaba el 10% de descuento de cliente frecuente a la tarifa de boleto perdido ($270 en lugar de los $300 fijos).

4. Una suite con 100% de pruebas aprobadas demuestra que el programa es correcto? Explique.

No. Que todas las pruebas pasen solo demuestra que el programa responde bien a los escenarios que decidimos probar. Por ejemplo, de los 17 casos que diseñamos en el cuadro pasaron todas las pruebas, pero si colocábamos un dato no numérico en el argumento de minutos, por el orden de la validación intentaba evaluar si era menor a 0 y rompía el programa.

5. Si una IA generará automáticamente 50 pruebas, que tendría que revisar una persona antes de confiar en ellas?

Una persona tendría que verificar y validar las pruebas generadas por la IA, que sus resultados sean correctos y relevantes y sigan la lógica y lo esperado del programa. Se debe verificar que no sean repetitivas o tengan una gran área ciega, que los resultados sigan las especificaciones del programa, además de que no estén generadas según el código y el programa, sino por los requerimientos y

especificaciones del programa y sean verdaderamente útiles revisando grandes o importantes áreas y funciones del programa.
Es decir, una persona debe verificar las 50 pruebas, entendiendo el programa, lo que se busca, la lógica y que serían buenas pruebas, ya que aunque la IA puede generar rápidamente y fácilmente las pruebas, la persona aun tiene la responsabilidad de revisar y verificar las pruebas.